

Mbeya University of Science and Technology

IT 8113: OBJECT ORIENTED PROGRAMMING

Facilitator: Mr. Magemo A

Email: magemohoward30@gmail.com

Lecture 2



Control Statement

if-else

If statement is used to test the condition. There are various types of if statements in C++.

1. if statement
2. if-else statement
3. nested if statement
4. if-else-if ladder

Control Statement

IF Statement

- if statement tests the condition. It is executed if condition is true.

Syntax

```
if(condition){  
  //code to be  
  executed  
}
```

Example

```
#include <iostream>  
using namespace std;
```

```
int main () {  
    int num = 10;  
        if (num % 2 == 0)  
        {  
            cout<<"It is even number";  
        }  
    return 0;  
}
```

Control Statement

IF-else Statement

if-else statement also tests the condition. It executes if block if condition is true otherwise else block is executed.

Syntax

```
if(condition){  
    //code if condition is  
    true  
} else{  
    //code if condition is  
    false  
}
```

Example

```
#include <iostream>  
using namespace std;  
int main () {  
    int num = 11;  
    if (num % 2 == 0)  
    {  
        cout<<"It is even number";  
    }  
    else  
    {  
        cout<<"It is odd number";  
    }  
    return 0;  
}
```

Control Statement

IF-else-if ladder Statement

if-else-if ladder statement executes one condition from multiple statements.

Syntax

```
if(condition1){  
    //code to be executed if condition1 is true  
}else if(condition2){  
    //code to be executed if condition2 is true  
}  
else if(condition3){  
    //code to be executed if condition3 is true  
}  
...  
else{  
    //code to be executed if all the conditions are false  
}
```

Control Statement

Example to check grade of students

```
#include <iostream>
using namespace std;
int main () {
    int num;
    cout<<"Enter a number to check grade:";
    cin>>num;
    if (num < 0 || num > 100)
    {
        cout<<"wrong number";
    }
    else if (num >= 0 && num < 50){
        cout<<"Fail";
    }
    else if (num >= 50 && num < 60)
    {
        cout<<"D Grade";
    }
    else if (num >= 60 && num < 70)
    {
        cout<<"C Grade";
    }
    else if (num >= 70 && num < 80)
    {
        cout<<"B Grade";
    }
    else if (num >= 80 && num < 90)
    {
        cout<<"A Grade";
    }
    else if (num >= 90 && num <= 100)
    {
        cout<<"A+ Grade";
    }
}
```

Control Statement

Switch statement

switch statement executes one statement from multiple conditions. It is like if-else-if ladder statement

Syntax

```
switch(expression){  
  case value1:  
    //code to be executed;  
    break;  
  case value2:  
    //code to be executed;  
    break;  
  .....  
  default:  
    //code to be executed if all cases are not matched;  
    break;  
}
```

Control Statement

Example

```
#include <iostream>
using namespace std;
int main () {
    int num;
    cout<<"Enter a number to check grade:";
    cin>>num;
    switch (num)
    {
        case 10: cout<<"It is 10"; break;
        case 20: cout<<"It is 20"; break;
        case 30: cout<<"It is 30"; break;
        default: cout<<"Not 10, 20 or 30"; break;
    }
}
```


Control Statement

For Loop

for loop is used to iterate a part of the program several times. If the number of iteration is fixed, it is recommended to use for loop than while or do-while loops.

We can initialize variable, check condition and increment/decrement value.

Syntax

```
or(initialization; condition; incr/decr)
{
    //code to be executed
}
```

Example

```
#include <iostream>
using namespace std;
int main() {
    for(int i=1;i<=10;i++){
        cout<<i <<"\n";
    }
}
```

Control Statement

While loop

while loop is used to iterate a part of the program several times. If the number of iteration is not fixed, it is recommended to use while loop than for loop.

Syntax

```
while(condition){  
    //code to be executed  
}
```

Example

```
#include <iostream>  
using namespace std;  
int main() {  
    int i=1;  
    while(i<=10)  
    {  
        cout<<i <<"\n";  
        i++;  
    }  
}
```

Control Statement

Do-While Loop

do-while loop is used to iterate a part of the program several times. If the number of iteration is not fixed and you must have to execute the loop at least once, it is recommended to use do-while loop.

The C++ do-while loop is executed at least once because condition is checked after loop body..

Syntax

```
do{  
    //code to be executed  
}while(condition);
```

Example

```
#include <iostream>  
using namespace std;  
int main() {  
    int i = 1;  
    do{  
        cout<<i<<"\n";  
        i++;  
    } while (i <= 10) ;  
}
```

Control Statement

Break Statement

break is used to break loop or switch statement. It breaks the current flow of the program at the given condition.

Syntax

```
jump-statement;  
break;
```

Example

```
#include <iostream>  
using namespace std;  
int main() {  
    for (int i = 1; i <= 10; i++)  
    {  
        if (i == 5)  
        {  
            break;  
        }  
        cout << i << "\n";  
    }  
}
```

Functions

- A function is a group of statements that together perform a task. Every C++ program has at least one function, which is `main()`, and all the most trivial programs can define additional functions.
- A **function declaration** tells the compiler about a **function's name**, **return type**, and **parameters**.
- A **function definition** provides the actual body of the function.



Functions

Defining a Function

```
return_type function_name( parameter list ) {  
    body of the function  
}
```

Function definition consists of a **function header** and a **function body**



Functions

Defining a Function

Return Type - A function may return a value. The return type is the data type of the value the function returns.

- Some functions perform the desired operations without returning a value. In this case, the return type is the keyword void.

Function Name - This is the actual name of the function. The function name and the parameter list together constitute the function signature.

Functions

Defining a Function

Parameters – A parameter is like a placeholder. When a function is invoked, you pass a value to the parameter. This value is referred to as actual parameter or argument.

- The parameter list refers to the type, order, and number of the parameters of a function. Parameters are optional; that is, a function may contain no parameters.

Function Body – The function body contains a collection of statements that define what the function does.

Functions

Example

The code for a function called max(). This function takes two parameters num1 and num2 and return the biggest of both -

```
// function returning the max between two numbers

int max(int num1, int num2) {
    // local variable declaration
    int result;

    if (num1 > num2)
        result = num1;
    else
        result = num2;

    return result;
}
```

Functions

Function Declarations

- A function declaration tells the compiler about a function name and how to call the function.
- A function declaration has the following parts -

```
return_type function_name( parameter list );
```

- For the above defined function max(), following is the function declaration -

```
int max(int num1, int num2);
```



Functions

Calling a Function

- When a program calls a function, program control is transferred to the called function.
- A called function performs defined task and when its return statement is executed or when its function-ending closing brace is reached, it returns program control back to the main program.
- To call a function, you simply need to pass the required parameters along with function name, and if function returns a value, then you can store returned value



Functions

Example

// Create a function

```
void myHello() {  
    cout << "I just got executed!";  
}
```

```
int main() {  
    myHello(); // call the function  
    return 0;  
}
```

Functions

Example

```
#include <iostream>
using namespace std;
```

```
// function declaration
int max(int num1, int num2);
```

```
int main () {
    // local variable declaration:
    int a = 100;
    int b = 200;
    int ret;

    // calling a function to get max value.
    ret = max(a, b);
    cout << "Max value is : " << ret << endl;
    return 0;
}
```

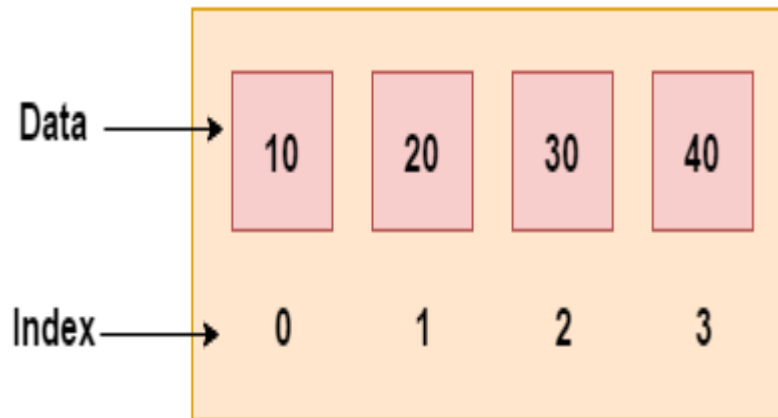
```
// function returning the max between
two numbers
int max(int num1, int num2) {
    // local variable declaration
    int result;

    if (num1 > num2)
        result = num1;
    else
        result = num2;

    return result;
}
```

◆ Arrays

- Array in C++ is a group of similar types of elements that have contiguous memory location.
- **C++ `std::array`** is a container that encapsulates fixed size arrays. In C++, array index starts from 0. We can store only fixed set of elements in C++ array.



Arrays

Declaring Arrays.

```
type arrayName [ arraySize ];
```

Example: `double balance[10];`

Initializing Arrays

```
double balance[5] = {1000.0, 2.0, 3.4, 17.0, 50.0};
```

Arrays

Advantages of C++ Array

1. Code Optimization (less code)
2. Random Access
3. Easy to traverse data
4. Easy to manipulate data
5. Easy to sort data etc.

Disadvantages of C++ Array

1. Fixed size

Arrays

Array Types

There are 2 types of arrays in C++ programming:

1. Single Dimensional Array
2. Multidimensional Array

Arrays

Single Dimensional Array- Example

```
#include <iostream>
using namespace std;
int main()
{
    int arr[5]={10, 0, 20, 0, 30}; //creating and
    initializing array
        //traversing array
        for (int i = 0; i < 5; i++)
        {
            cout<<arr[i]<<"\n";
        }
}
```

Arrays

Passing Array to Function Example: print array elements

```
#include <iostream>
using namespace std;
void printArray(int arr[5]);
int main()
{
    int arr1[5] = { 10, 20, 30, 40, 50 };
    int arr2[5] = { 5, 15, 25, 35, 45 };
    printArray(arr1); // passing array to function

    printArray(arr2);
}
```

```
void printArray(int arr[5])
{
    cout << "Printing array elements:" <<
    endl;
    for (int i = 0; i < 5; i++)
    {
        cout << arr[i] << "\n";
    }
}
```

Arrays

Passing Array to Function Example: Print minimum number

```
#include <iostream>
using namespace std;
void printMin(int arr[5]);
int main()
{
    int arr1[5] = { 30, 10, 20, 40, 50 };
    int arr2[5] = { 5, 15, 25, 35, 45 };
    printMin(arr1); // passing array to function

    printMin(arr2);
}
```

```
void printMin(int arr[5])
{
    int min = arr[0];
    for (int i = 0; i < 5; i++)
    {
        if (min > arr[i])
        {
            min = arr[i];
        }
    }
    cout << "Minimum element is: " <<
    min << "\n";
}
```

Arrays

Multidimensional Array

Multidimensional array in C++ declares, initializes and traverse two dimensional arrays.

```
#include <iostream>

using namespace std;

int main()
{
    int sale[2][3]={{3,5,7},{2,10,8}};
    cout<<sale [0][2];
}
```

Output

7

Arrays

Multidimensional Array

To print all element

```
#include <iostream>
using namespace std;

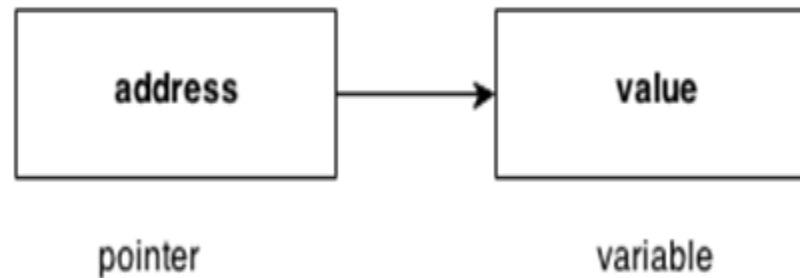
int main()
{
    int sale[2][3]={{3,5,7},{2,10,8}};
    for(int i=0;i<2;i++){
        for(int j=0;j<3;j++){
            cout<<sale[i][j];
            cout<<endl;
        }
    }
}
```

Output

```
3 5 7
2 10 8
```

◆ Pointers

- The pointer in C++ language is a variable, it is also known as locator or indicator that points to an address of a value.



Pointers

Advantage of pointer

1. Pointer reduces the code and improves the performance
2. We can return multiple values from function using pointer.
3. It makes you able to access any memory location in the computer's memory.

Pointers

Declaring a pointer

The pointer in C++ language can be declared using * (asterisk symbol).

1. `int * a; //pointer to int`

2. `char * c; //pointer to char`

Pointers

Example of using pointers printing the address and value.

```
#include <iostream>
using namespace std;
int main()
{
    int number=30;
    int * p;
    p=&number;//stores the address of number variable
    cout<<"Address of number variable is:"<<&number<<endl;
    cout<<"Address of p variable is:"<<p<<endl;
    cout<<"Value of p variable is:"<<*p<<endl;
    return 0;
}
```

Pointers

Example of using pointers printing the address and value.

Output

Address of number variable is:0x7ffccc8724c4

Address of p variable is:0x7ffccc8724c4

Value of p variable is:30

Array of Pointers

- Array and pointers are closely related to each other.
- In C++, the name of an array is considered as a pointer, i.e., the name of an array contains the address of an element.
- C++ considers the array name as the address of the **first element**
- Therefore, we can say that array name is a pointer which is holding the address of the first element of an array.



Array of Pointers

Example

```
#include <iostream>
using namespace std;
int main()
{
    int *ptr; // integer pointer declaration
    int marks[10]; // marks array declaration
    cout << "Enter the elements of an array :" << endl;
    for(int i=0;i<10;i++)
    {
        cin>>marks[i];
    }
    ptr=marks; // both marks and ptr pointing to the same element..
    cout << "The value of *ptr is :" <<*ptr<< endl;
    cout << "The value of *marks is :" <<*marks<<endl;
}
```

ASSIGNMENT

Write the program from the following information

1. By passing array to a function write a program to display maximum marks of a students entered by a facilitator
2. Write a program that asks the user to enter string name and integers year as inputs to be stored in the variables two variable respectively. There are also two integer pointers. Assign the values of string and integer to first pointer and second pointer respectively, and display them.

Thank you for Listening! Questions

